Федеральное государственное автономное образовательное учреждение высшего образования

«Национальный исследовательский университет ИТМО»

Факультет программной инженерии и компьютерной техники

Отчёт по задачам раздела №4

Алгоритмы на графах

Выполнил: Караганов Павел

Группа: P3210

Проверил: Косяков Михаил Сергеевич

г. Санкт-Петербург,

2026

M. Цивилизация

Задача решается поиском кратчайшего пути на сетке с различной стоимостью входа в клетку: `.` имеет вес `1`, `W` — `2`, а препятствия считаются непроходимыми. Для этого используется алгоритм Дейкстры с восстановлением маршрута.

Идея реализации:
- хранение расстояний `d` до клеток и предков `p` для восстановления пути;
- выбор вершины с минимальным текущим расстоянием через `set<pair<int, Point>>` (здесь используется ключевое свойство алгоритма Дейкстры: если вершина извлечена как минимальная, то кратчайшее расстояние до неё уже найдено);
- рассмотрение четырёх направлений (север, восток, юг, запад).

Асимптотика:
- время: $O((n * m) \log(n * m))$;
- память: $O(n * m)$.

Почему так:
- в графе сетки не более $V = n * m$ вершин и $E = O(n * m)$ рёбер (до четырёх на вершину);
- Дейкстра с очередью на основе сбалансированного дерева выполняется за $O((V + E) \log V)$;
- подставляя оценки для сетки, получаем $O((n * m) \log(n * m))$;
- массивы `d`, `p` и поле имеют размер порядка $n * m$, поэтому память линейна по числу клеток.

Почему это оптимальное решение:
- веса рёбер неотрицательные и различаются (1 и 2), поэтому задача естественно сводится к поиску кратчайшего пути, где Дейкстра даёт корректный результат;
- алгоритм одновременно находит минимальную стоимость и позволяет восстановить сам маршрут через массив предков;

NB: После завершения Дейкстры маршрут восстанавливается по массиву предков от финиша к старту, затем разворачивается в направлении движения символами `N`, `E`, `S`, `W`.

```
#include <algorithm>
#include <deque>
#include <iostream>
#include <map>
#include <queue>
#include <set>
#include <stack>
#include <string>
#include <vector>

#define INF 1e9
#define ll long long

using namespace std;

struct Point {
  int x;
  int y;

  Point() {
    this->x = 0;
    this->y = 0;
```

```
  }

  Point(int x, int y) {
    this->x = x;
    this->y = y;
  }

  bool operator!=(const Point& o) const {
    return !(x == o.x && y == o.y);
  }

  bool operator==(const Point& o) const {
    return x == o.x && y == o.y;
  }

  bool operator<(const Point& o) const {
    if (x != o.x)
      return x < o.x;
    return y < o.y;
  }

  Point& operator--() {
    --x;
    --y;
    return *this;
  }

  Point operator--(int) {
    Point cur = *this;
    --x;
    --y;
    return cur;
  }
};

int weight(char x) {
  if (x == '.') {
    return 1;
  } else if (x == 'W') {
    return 2;
  } else {
    return INF;
  }
}

int main() {
  int n, m;
  Point s, f;

  cin >> n >> m >> s.x >> s.y >> f.x >> f.y;

  --s;
  --f;

  vector<vector<char>> g(n, vector<char>(m, 0));

  for (int i = 0; i < n; i++) {
```

```
    for (int j = 0; j < m; j++) {
      cin >> g[i][j];
    }
  }

  vector<vector<int>> d(n, vector<int>(m, INF));
  vector<vector<Point>> p(n, vector<Point>(m, {-1, -1}));
  set<pair<int, Point>> set;

  d[s.x][s.y] = 0;
  set.insert({0, s});

  while (!set.empty()) {
    Point v = (*set.begin()).second;
    set.erase(set.begin());

    if (v == f) {
      break;
    }

    if (v.x - 1 >= 0 &&
        d[v.x][v.y] + weight(g[v.x - 1][v.y]) < d[v.x - 1][v.y]) {  // можно ли идти
вверх
      set.erase({d[v.x - 1][v.y], Point(v.x - 1, v.y)});
      d[v.x - 1][v.y] = d[v.x][v.y] + weight(g[v.x - 1][v.y]);
      p[v.x - 1][v.y] = v;
      set.insert({d[v.x - 1][v.y], Point(v.x - 1, v.y)});
    }

    if (v.y + 1 < m &&
        d[v.x][v.y] + weight(g[v.x][v.y + 1]) < d[v.x][v.y + 1]) {  // можно ли идти
вправо
      set.erase({d[v.x][v.y + 1], Point(v.x, v.y + 1)});
      d[v.x][v.y + 1] = d[v.x][v.y] + weight(g[v.x][v.y + 1]);
      p[v.x][v.y + 1] = v;
      set.insert({d[v.x][v.y + 1], Point(v.x, v.y + 1)});
    }

    if (v.x + 1 < n &&
        d[v.x][v.y] + weight(g[v.x + 1][v.y]) < d[v.x + 1][v.y]) {  // можно ли идти
вниз
      set.erase({d[v.x + 1][v.y], Point(v.x + 1, v.y)});
      d[v.x + 1][v.y] = d[v.x][v.y] + weight(g[v.x + 1][v.y]);
      p[v.x + 1][v.y] = v;
      set.insert({d[v.x + 1][v.y], Point(v.x + 1, v.y)});
    }

    if (v.y - 1 >= 0 &&
        d[v.x][v.y] + weight(g[v.x][v.y - 1]) < d[v.x][v.y - 1]) {  // можно ли идти
влево
      set.insert({d[v.x][v.y - 1], Point(v.x, v.y - 1)});
      d[v.x][v.y - 1] = d[v.x][v.y] + weight(g[v.x][v.y - 1]);
      p[v.x][v.y - 1] = v;
      set.insert({d[v.x][v.y - 1], Point(v.x, v.y - 1)});
    }
  }
```

```
  if (d[f.x][f.y] == INF) {
    cout << -1;
    return 0;
  } else {
    cout << d[f.x][f.y] << '\n';

    vector<Point> path;
    Point to, from;

    for (Point v = f; v != s; v = p[v.x][v.y]) {
      path.push_back(v);
    }

    path.push_back(s);

    for (int i = (int)path.size() - 1; i > 0; i--) {
      from = path[i];
      to = path[i - 1];

      if (from.y == to.y && from.x - 1 == to.x) {
        cout << 'N';
      } else if (from.x == to.x && from.y + 1 == to.y) {
        cout << 'E';
      } else if (from.y == to.y && from.x + 1 == to.x) {
        cout << 'S';
      } else {
        cout << 'W';
      }
    }
  }
}
```

N. Свинки-копилки

Построен неориентированный граф соответствий: каждая вершина соединяется с указанной для неё вершиной. Ответ равен числу компонент связности, которое находится обходом в глубину.

Идея реализации:
- построение списка смежности;
- запуск DFS из каждой ещё не посещённой вершины;
- увеличение счётчика компонент при каждом новом запуске DFS.

Асимптотика:
- время: $O(n)$;
- память: $O(n)$.

Почему так:
- для каждой из n вершин добавляется по одному соответствию, то есть суммарно рёбер порядка n;
- DFS обходит каждую вершину и каждое ребро константное число раз;
- значит суммарная сложность линейная: $O(n)$;
- списки смежности и массив `used` занимают линейную память.

Почему это оптимальное решение:

- чтобы посчитать число компонент связности, необходимо как минимум просмотреть все вершины и рёбра, то есть нижняя граница по времени линейная;
- DFS достигает этой границы, работая за $O(n)$ в данном графе;

NB: В этой задаче можно игнорировать ориентацию рёбер, так как исходный граф функциональный (у каждой вершины ровно одно исходящее ребро). В каждой компоненте связности такого графа существует ровно один направленный цикл. Чтобы открыть все копилки в компоненте, достаточно разбить любую копилку на этом цикле, после чего остальные откроются по ключам вдоль рёбер. Следовательно, минимальное число разбитых копилок равно числу таких циклов, а оно совпадает с числом компонент связности неориентированного графа.

```
#include <iostream>
#include <string>
#include <vector>
#define INF INT_MAX
#define ll long long

using namespace std;

void dfs(int v, vector<vector<int>>& graph, vector<bool>& used) {
  used[v] = true;

  for (int to : graph[v]) {
    if (!used[to]) {
      dfs(to, graph, used);
    }
  }
}

int main() {
  int n;
  cin >> n;

  int component = 0;

  int edge;

  vector<vector<int>> graph(n);
  vector<bool> used(n, false);

  for (int i = 0; i < n; i++) {
    cin >> edge;

    edge--;
    graph[i].push_back(edge);
    graph[edge].push_back(i);
  }

  for (int i = 0; i < n; i++) {
    if (!used[i]) {
      dfs(i, graph, used);
      component++;
    }
  }
```

```
  cout << component;
}
```

O. Долой списывание!

Задача сводится к проверке двудольности графа конфликтов: если граф можно раскрасить в два цвета, то корректное разделение возможно, иначе нет. Используется DFS-раскраска.

Идея реализации:
- хранение цветов вершин (`0` — не окрашена, `1` и `2` — доли);
- DFS с присваиванием соседям противоположного цвета;
- при конфликте цветов фиксируется невозможность разбиения.

Асимптотика:
- время: $O(n + k)$, где n — число вершин, k — число рёбер;
- память: $O(n + k)$.

Почему так:
- построение графа по списку рёбер занимает $O(k)$;
- обход DFS посещает каждую вершину один раз и просматривает каждое ребро не более двух раз;
- суммарно получаем $O(n + k)$;
- хранение списка смежности и массива цветов требует $O(n + k)$ памяти.

Почему это оптимальное решение:
- условие задачи напрямую эквивалентно двудольности графа, поэтому проверка двухцветной раскраски является самым естественным критерием;
- DFS-раскраска проверяет двудольность за линейное время по размеру графа, что асимптотически оптимально;
- при этом сразу получается конструктивная раскраска, если разбиение возможно.

NB: Итог `YES` выводится тогда и только тогда, когда в графе нет нечётных циклов, что эквивалентно двудольности.

```
#include <iostream>
#include <map>
#include <stack>
#include <string>
#include <vector>
#define INF INT_MAX
#define ll long long

using namespace std;

bool proigrali = false;

void dfs(int v, int c, vector<vector<int>>& g, vector<int>& color) {
  color[v] = c;
  for (int to : g[v]) {
    if (color[to] == 0) {
      dfs(to, 3 - c, g, color);
    } else if (color[to] == c) {
```

```
      proigrali = true;
    }
  }
}

int main() {
  int n, k;
  cin >> n >> k;
  int to, from;

  vector<vector<int>> graph(n);
  vector<int> color(n, 0);

  for (int i = 0; i < k; i++) {
    cin >> from >> to;
    graph[--from].push_back(--to);
    graph[to].push_back(from);
  }

  for (int i = 0; i < n; i++) {
    if (color[i] == 0) {
      dfs(i, 1, graph, color);
    }
  }

  cout << (proigrali ? "NO" : "YES");
}
```

P. Авиаперелёты

Ищется минимальный порог стоимости x, при котором из любой вершины можно добраться в любую другую, используя только рёбра с весом не больше x. Применяется бинарный поиск по ответу и проверка сильной связности в исходном и обратном графах.

Идея реализации:
- бинарный поиск по диапазону возможных весов (от самого маленького по весу ребра до самого большого);
- функция `check(x)` запускает DFS по рёбрам `<= x` вперёд и назад;
- если обе проверки покрывают все вершины, порог достаточен и мы имеем 1 КСС.

Асимптотика:
- время: $O(n^2 \log C)$, где $C = \max_w - \min_w + 1$;
- память: $O(n^2)$.

Почему так:
- граф задан матрицей смежности $n \times n$;
- один DFS при таком представлении перебирает всю строку матрицы для каждой вершины, то есть $O(n^2)$;
- в `check(x)` выполняются два DFS, что сохраняет порядок $O(n^2)$;
- бинарный поиск делает $O(\log C)$ проверок;
- итог: $O(n^2 \log C)$, а хранение матрицы весов требует $O(n^2)$ памяти.

Почему это оптимальное решение:
- монотонность условия по порогу x (если связно при x, то связно и при большем пороге) делает бинарный поиск по ответу корректным и эффективным;

- проверка сильной связности через обход исходного и обратного графов является стандартным необходимым и достаточным критерием на проверку КСС;
- при матричном представлении графа стоимость одной проверки неизбежно порядка $O(n^2)$, поэтому итоговая сложность решения близка к наилучшей для выбранного формата входа.

```
#include <algorithm>
#include <deque>
#include <iostream>
#include <map>
#include <queue>
#include <stack>
#include <string>
#include <vector>

#define INF 1e9
#define ll long long

using namespace std;

int n;
int component = 0;

void dfs(int v, int x, vector<vector<int>>& g, vector<bool>& used) {
  used[v] = true;

  for (int to = 0; to < n; to++) {
    if (!used[to] && g[v][to] <= x) {
      dfs(to, x, g, used);
    }
  }
}

void dfs_revers(int v, int x, vector<vector<int>>& g, vector<bool>& used) {
  used[v] = true;

  for (int from = 0; from < n; from++) {
    if (!used[from] && g[from][v] <= x) {
      dfs_revers(from, x, g, used);
    }
  }
}

bool check(int x, vector<vector<int>>& g) {
  vector<bool> used(n, false);
  dfs(0, x, g, used);
  for (int i = 0; i < n; i++) {
    if (!used[i]) {
      return false;
    }
  }

  for (int i = 0; i < (int)used.size(); i++) {
    used[i] = false;
  }

  dfs_revers(0, x, g, used);
  for (int i = 0; i < n; i++) {
```

```
    if (!used[i]) {
      return false;
    }
  }

  return true;
}

void bin_search(ll l, ll r, vector<vector<int>>& g) {
  if (r - l < 2) {
    cout << r;
    return;
  }

  int mid = (l + r) / 2;
  check(mid, g) ? bin_search(l, mid, g) : bin_search(mid, r, g);
}

int main() {
  cin >> n;
  vector<vector<int>> g(n, vector<int>(n, 0));

  ll minim = INF;
  ll maxim = 0;

  for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
      cin >> g[i][j];
      if (i != j) {
        minim = min(minim, (ll)g[i][j]);
        maxim = max(maxim, (ll)g[i][j]);
      }
    }
  }

  bin_search(minim, maxim, g);
}
```